

喊话器协议

连接

ip: 192.168.144.38 (MK15, H16)

端口: 8519

协议类型: tcp

设置音量

协议类型: tcp

请求数据:[14]\${vol}

参数说明：

 \${vol} 指音量大小 取值范围 0-100， 使用 16 进制数据传输，小于 16 的数据需要

 前置补齐 0x00

示例: [14]64

音量状态定时上报：

返回数据样例：[99]"{"volume_real": 58, "volume_limit": 100, "temperature": "23.56"}"

每 2s 自动上报一条 ,volume_real 是实际生效音量 ,volume_limit 是限制能达到的最大音量，
temperature 是温度，三个字段都是十进制数字。这个定时上报消息只在单独的喊话器和四
合一喊话器中有。

获取录音文件列表

协议类型: http

请求: http://[负载 ip]:8222/fetch-files

示例: http://192.168.144.27:8222/fetch-files

```
返回数据: {  
  code: 0,  
  data: []  
}
```

成功时, code=0, data 是获取到的文件名列表

实时喊话

协议类型: tcp

请求数据 (移动端使用): [10]\${data}

移动端在停止喊话的时候需要发送: [11]

请求数据 (PC 端使用): [42]\${data}

参数说明:

 \${data} 音频数据, 通过 opus 编码后的音频数据, 采样率 8000, 通道数 1, 帧大小 480

上传音频文件

音频格式目前支持: wav, mp3, aac, m4a, flac

协议类型: http

请求数据: http://[负载 ip]:8222/upload-file

示例：

```
import okhttp3.*;
import okhttp3.MediaType;
import okhttp3.RequestBody;
import okhttp3.MultipartBody;
import org.json.JSONObject;
import java.io.File;
import java.io.IOException;
public class YourClass {
    private static final String TAG = "YourClass";
    private OkHttpClient client = new OkHttpClient();
    private static final MediaType MEDIA_TYPE_MARKDOWN =
    MediaType.parse("text/x-markdown; charset=utf-8");

    public boolean uploadFile(File file, ProgressRequestBody.ProgressCallback callback) {
        MultipartBody.Builder requestBody = new
        MultipartBody.Builder().setType(MultipartBody.FORM); // 文件和 json 参数共同上传

        if (file != null) { // 添加文件到 form-data
            RequestBody body = RequestBody.create(MEDIA_TYPE_MARKDOWN, file);
            // 参数分别为，请求 key，文件名称，RequestBody
            requestBody.addFormDataPart("file", file.getName(), body);
        }

        Request request = new Request.Builder()
            .url("http://192.168.144.27:8222/upload-file")
            .post(requestBody.build())
            .build();

        try (Response response = client.newCall(request).execute()) {
            JSONObject jsonResp = new JSONObject(response.body().string());
            if (jsonResp.getInt("code") == 0) {
                return true;
            }
            Log.i(TAG, "错误代号:" + jsonResp.getInt("code"));
        } catch (IOException e) {
            e.printStackTrace();
        }
        return false;
    }
}
```

播放录音

协议类型: tcp

请求数据:[12]\${loop}\${name}

参数说明：

 \${name} 文件名 utf8 编码

 \${loop} 是否循环播放, 循环 1，单次 0

停止播放录音

协议类型: tcp

请求数据:[13]

删除录音

协议类型: http

请求数据:http://[负载 ip]:8222/del-file

Content-Type = "application/x-www-form-urlencoded"

参数说明：{

 filename: 文件名

}

返回数据：{

 code:0

}

code=0 时，操作成功

文字转语音

协议类型: tcp

请求数据:[15]\${text}

参数说明：

 \${text} 要播放的文字内容

文字转语音循环播放

协议类型: tcp

请求数据:[16]\${text}

参数说明：

 \${text} 要播放的文字内容

停止文字转语音循环播放

协议类型: tcp

请求数据:[17]

文字转语音 V2

支持男女生

请求数据[31]\${voice}\${text}

参数说明：

`${voice}` 男女声 0 男声 1 女声

`${text}` 要播放的文字内容

文字转语音循环播放 V2

支持男女生

请求数据[32]`${voice}${text}`

参数说明：

`${voice}` 男女声 0 男声 1 女声

`${text}` 要播放的文字内容

播放警报

协议类型: tcp

请求数据:[18]

停止播放警报

协议类型: tcp

请求数据:[19]

俯仰控制

协议类型 : tcp

端口 : 12345

请求数据：8D \${俯仰值}

例如：8D 5A，两个字节，第一位是固定值 8D，第二位是俯仰值，俯仰值是 80-220 之间的无符号整数

收音

协议类型：tcp

1. 发送开始收音请求：[40]
2. 监听，接收到收音数据：[40]\${音频}，是一个 ByteArray，前四位是[40]，后面接的是使用 Opus 编码的音频数据，需要解码为 PCM 格式后播放。音频采样率为 16000，单声道，每帧解码后的大小不超过 320 字节（20ms 帧）。
3. 发送停止收音请求：[41]

获取自定义 sn

协议类型: tcp

请求数据:[89]

返回示例：

[89]" 172a783135658ac7"

获取主控芯片唯一 id

协议类型: tcp

请求数据:[90]

返回示例：

[90]" e3deacfd02b1b686"

